CD SECURITY

AUDIT REPORT

Tria

January 2026

Prepared by
ZanyBonzy
ArnieSec
zeroXchad

Introduction

A time-boxed security review of the **Tria** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About **Tria**

Tria is a token claiming and staking protocol that enables users to claim token allocations via Merkle proofs and stake them for rewards. The protocol consists of three core contracts: **TriaClaim** handles Merkle-based token claiming with a configurable fee structure and supports both direct claims and allocations through staking/spending; **TriaStakingPool** manages user deposits with time-locked withdrawals, FIFO-based reward calculations, and emergency withdrawal capabilities; and **TriaSpend** governs spending allocations and token whitelisting across the system.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

***review commit hash* - 895f75b0388b007cb8a5f2640bdb5f972723b4db**

***fixes review commit hash* - 802b0cb9a1558c73ffcb0a626c170fbf1491b0e0**

Scope

The following smart contracts were in scope of the audit:

- `TriaClaim.sol`
- `TriaStakingPool.sol`
- `TriaSpend.sol`

The following number of issues were found, categorized by their severity:

- Critical & High: 0 issues
- Medium: 4 issues
- Low: 4 issues
- Informational: 4 issues

Findings Summary

ID	Title	Severity	Status
[M-01]	Deposit allows for DOS of victim recipient	Medium	Acknowledged
[M-02]	`claimDirect` is missing `directClaimEnabled` check	Medium	Acknowledged
[M-03]	Users can bypass fees by allocating all to staking and emergency withdrawing immediately	Medium	Acknowledged
[M-04]	Deposit Queue is not cleared during withdrawals, increasing DOS risks for future deposit	Medium	Acknowledged
[L-01]	Missing Validation Between `rewardEndTimestamp` and `globalUnlockTimestamp` Allows Locking Users Without Rewards	Low	Acknowledged
[L-02]	`checkWithdrawSolvency` Returns False Positive When Pool Is Insolvent For Principal	Low	Fixed
[L-03]	`_withdraw` can revert due to arithmetic underflow	Low	Fixed
[L-04]	`canAcceptClaims` can return misleading result	Low	Fixed
[I-01]	Duplicate condition in `_getWithdrawableBalance` and `checkWithdrawSolvency` functions	Informational	Fixed
[I-02]	Gaps are not needed	Informational	Acknowledged
[L-03]	Partial Withdrawals Enabled Despite Documentation Stating Otherwise	Informational	Fixed
[I-04]	Remove Redundant Checks	Informational	Fixed

Detailed Findings

[M-01] Deposit allows for DOS of victim recipient

Severity

Impact: High

Likelihood: Low

Description

```
function deposit(
    address _recipient,
    uint256 _amount
)
```

When depositing, users will not be able to deposit if the `currentDepositCount` exceeds `maxDepositsForUser`. Since `deposit` allows any user to deposit on behalf of another, a user can deposit the minimum amount up until `maxDepositsForUser` is reached for the recipient address.

```
if (currentDepositCount >= config.maxDepositsPerUser)
    revert MaxDepositsReached(
        currentDepositCount,
        config.maxDepositsPerUser
    );
```

The recipient user will no longer be able to make any additional deposits under the account. This will result in a denial of service for the recipient address.

Recommendations

Consider restricting deposits for another recipient to be limited to the `TriaClaim` contract.

[M-02] `claimDirect` is missing `directClaimEnabled` check

Severity

Impact: Medium

Likelihood: High

Description

`directClaimEnabled` is a boolean flag that determines whether users can claim tokens directly to their wallet or through the stake/spend allocation system. When `directClaimEnabled` is set to `false`, it is

intended that users have to only claim via the spend allocation system.

```
    /// @notice Enables or disables direct claim mode
    /// @dev When enabled, spend allocation goes directly to user instead
of Spend contract
    /// @param _enabled Whether direct claim should be enabled
    function setDirectClaimEnabled(bool _enabled) external onlyOwner {
        directClaimEnabled = _enabled;
        emit DirectClaimEnabledUpdated(_enabled);
    }
```

However, the `claimDirect` function lacks this check, allowing users to bypass the intended flow by claiming directly to their wallet regardless of `directClaimEnabled`'s status.

Recommendations

Add a check at the beginning of the `claimDirect` function to ensure that `directClaimEnabled` is `true` before allowing the claim to proceed.

[M-03] Users can bypass fees by allocating all to staking and emergency withdrawing immediately

Severity

Impact: High

Likelihood: Low

Description

Direct claiming without staking in TriaClaim.sol follows the steps below:

1. User calls `claim`/`claimDirect` function
2. User pays fees
3. In case of `claim`: User receives tokens directly to their wallet or in the TriaSpend contract
4. In case of `claimDirect`: User receives tokens directly to their wallet

But users can bypass step 2 and 3 by claiming via `claim` with 100% `_stakeAllocation` and then calling `emergencyWithdraw` immediately in the staking pool.

No fees are charged when `_spendAllocation` is 0, and users that claim directly are presumably not interested in the staking reward system. Since `emergencyWithdraw` allows immediate withdrawal of full principal, only forfeiting rewards, such users can bypass the fee structure, bypass the `directClaimEnabled` check and still suffer no consequences matching their initial intent.

Recommendations

Recommend introducing a more severe penalty in `emergencyWithdraw` beyond reward forfeiture, e.g a percentage of principal being sent to the fee recipient or distributed as rewards.

[M-04] Deposit Queue is not cleared during withdrawals, increasing DOS risks for future deposit

Severity

Impact: High

Likelihood: Low

Description

`deposit` in TriaStakingPool sets the `Deposit` struct and pushes it into the `userDeposits` array. This array has a maximum value of `maxDepositsPerUser` which the user must not exceed. However, when a user withdraws, the empty deposit structs are not removed from the array. which as a result would lead to two issues:

1. Once deposit limit is reached, even if all deposits have been withdrawn, user cannot deposit more as the `userDeposits` array is now full.
2. If `maxDepositsPerUser` array is increased, the risk of out of gas error for such users increases as loop becomes larger.

We can test this by adding the test below to any of the staking pool's test files:

```
    /// @notice Withdraw exactly withdrawable succeeds
    function test_Amount_WithdrawDepositDos() public {

    // We set max deposit per user to 1 for test purposes;
        vm.prank(owner);
        pool.setMaxDepositsPerUser(1);

    // User deposits;
        vm.prank(user1);
        pool.deposit(user1, 1000 ether);

    // User tries to deposit again, should fail, showing that max deposit
is indeed 1;
        vm.prank(user1);
        vm.expectRevert();
        pool.deposit(user1, 1000 ether);

    // We skip 31 days (@note this can also be done by direct
`emergencyWithdraw` call);
        vm.warp(block.timestamp + 30 days + 1);

    // We fully withdraw user's balance;
        uint256 withdrawable = pool.getWithdrawableBalance(user1);
```

```
        vm.prank(user1);
        pool.withdraw(user1, withdrawable);

    // We ensure that user's balance is now 0;
        assertEq(pool.getTotalUserBalance(user1), 0);

    // User tries to deposit again, it fails;
        vm.prank(user1);
        vm.expectRevert();
        pool.deposit(user1, 1000 ether);

    }
```

Invariant #4 is also broken.

From docs:

> INV-4: After withdraw of deposit[i]: user.deposits[i] is removed (FIFO) "Withdrawn deposits are deleted, not zeroed"

Recommendations

Consider implementing a mechanism to clear the deposit queue during direct and emergency withdrawals.

[L-01] Missing Validation Between `rewardEndTimestamp` and `globalUnlockTimestamp` Allows Locking Users Without Rewards

Severity

Impact: Low

Likelihood: Low

Description

The contract allows `rewardEndTimestamp` and `globalUnlockTimestamp` to be set independently without validation of each other. This creates a scenario where users can be locked in the staking pool while rewards have stopped accruing.

`setGlobalUnlockTimestamp:`

```
function setGlobalUnlockTimestamp(uint256 _timestamp) external onlyOwner {
    if (_timestamp < block.timestamp) {
        revert GlobalUnlockTimestampInPast(_timestamp, block.timestamp);
    }
    uint256 oldTimestamp = globalUnlockTimestamp;
```

```
    globalUnlockTimestamp = _timestamp;
    emit GlobalUnlockTimestampUpdated(oldTimestamp, _timestamp);
}
```

`setRewardEndTimestamp:`

```
function setRewardEndTimestamp(uint256 _timestamp) external onlyOwner {
    if (_timestamp < block.timestamp) {
        revert RewardEndTimestampInPast(_timestamp, block.timestamp);
    }
    uint256 oldTimestamp = rewardEndTimestamp;
    rewardEndTimestamp = _timestamp;
    emit RewardEndTimestampUpdated(oldTimestamp, _timestamp);
}
```

Neither function validates the relationship between these two timestamps.

Recommendations

Add validation to ensure rewards continue accruing until at least the global unlock timestamp.

[L-02] `checkWithdrawSolvency` Returns False Positive When Pool Is Insolvent For Principal

Severity

Impact: Low

Likelihood: Low

Description

The `checkWithdrawSolvency` function only validates reward solvency but ignores principal solvency. When `balance` < `totalStaked`, the function can return `canWithdraw` = `true` even though the actual withdrawal would revert.

```
uint256 availableRewards = balance > totalStaked ? balance - totalStaked :
0;

if (pendingRewards <= availableRewards) {
    canWithdraw = true;  // Returns true even if balance < totalStaked
    rewardShortfall = 0;
}
```

If a user has zero pending rewards and the pool balance is less than `totalStaked`, the check 0 <= 0 passes and returns true. However, when the user attempts to withdraw, the calculation in `_withdraw` performs `balanceOf(this)` - `totalStaked` which underflows and reverts with a panic.

This misleads integrators and users relying on this view function to determine if withdrawals will succeed.

Recommendations

Add a principal solvency check before evaluating reward solvency:

```
uint256 balance = stakingToken.balanceOf(address(this));

if (balance < totalStaked) {
    return (false, totalStaked - balance);
}
```

[L-03] `_withdraw` can revert due to arithmetic underflow

Severity

Impact: Low

Likelihood: Low

Description

Within the `_withdraw` function a subtraction is done between contract's balance and `totalStaked`. This operation can cause transaction to revert in certain instance, when `adminWithdraw` was called in prior, and balance is actually lower than value saved in the `totalStaked` state parameter. As an effect, the function would not reach the actual revert with named error: `InsufficientRewardReserve` and can confuse end user with unhandled exception.

```
        // Check solvency
        uint256 availableRewards = stakingToken.balanceOf(address(this)) -
            totalStaked; //@audit can underflow
        if (totalRewards > availableRewards) {
            revert InsufficientRewardReserve(totalRewards,
availableRewards);
        }
```

Recommendations

It is recommended to add conditional check that handles the aforementioned scenario.

[L-04] `canAcceptClaims` can return misleading result

Severity

Impact: Low

Likelihood: Low

Description

The `canAcceptClaims` check several flags and properties to define whether the contract is ready to start claiming by users. However, the `isWhitelisted` flag can be misleading and irrelevant. Whenever all other conditions are met, the `claim` can be called with a configuration that transfer all tokens to the staking. In such case, the spend allocation is not checked at all. Thus, it is possible to call `claim` function despite the `canAcceptClaims` returns false.

```
        hasRoot = merkleRoot != bytes32(0);
        hasFeeBands = feeBandCount > 0;

        // When directClaimEnabled, spend contract is bypassed for spend
allocations
        // so whitelist check is not required for claims to succeed
        isWhitelisted = directClaimEnabled ||
spend.whitelisted(address(this));

        // Token consistency should always be verified - this is a
configuration check
        tokensMatch =
            (stakingPool.stakingToken() == address(token)) &&
            (spend.token() == address(token));

        ready = hasRoot && hasFeeBands && isWhitelisted && tokensMatch;
```

Recommendations

It is recommended to review the implementation of `canAcceptClaims` function and consider, whether the conditions included within it meet the desired business rules implemented in `claim` function.

[I-01] Duplicate condition in `_getWithdrawableBalance` and `checkWithdrawSolvency` functions

Description

The `dep.amount > 0` condition is checked twice within the `_getWithdrawableBalance` and `checkWithdrawSolvency` functions. Firstly in if condition, secondly, within the `_isDepositUnlocked` function. Thus, it consumes additional Gas during deployment and runtime.

Recommendations

It is recommended to remove duplicate condition and save some Gas.

[I-02] Gaps are not needed

Description

Each contract in the protocol is upgradable, however any of them is inherited. Yet, in every case gaps are implemented for future updates. Such implementation is redundant and consumes additional Gas for storage allocation.

```
    /// @notice Storage gap for future upgrades
    uint256[50] private __gap;
```

Recommendations

It is recommended to remove redundant gaps declaration.

[I-03] Partial Withdrawals Enabled Despite Documentation Stating Otherwise

Description

The `AUDIT_GUIDE` documentation explicitly states that partial withdrawals are not allowed, citing FIFO simplification and gaming prevention as rationale:

| No partial withdrawal | Must withdraw entire unlocked amount | Simplifies FIFO logic, prevents gaming |

However, the `_withdraw` function permits partial withdrawals from individual deposits:

```
} else {
    // Partial withdrawal from this deposit
    totalRewards += _calculateReward(dep, remaining);
    dep.amount = depositAmount - remaining;
    remaining = 0;
}
```

Users can call `withdraw` with any amount up to their withdrawable balance, and the function will partially consume deposits as needed.

Recommendations

If partial withdraws are truly not to be allowed, update the logic to reflect this.

[I-04] Remove Redundant Checks

Description

`claim` function has a check for `merkleRoot == bytes32(0)` which will never be true since `merkleRoot` can never be 0 as can be seen in the `initialize` and `setMerkleRoot` functions.

```
    function claim(
//...
        // Check merkle root is set
        if (merkleRoot == bytes32(0)) revert MerkleRootNotSet();
//...
```

`canAcceptClaims` function has checks for `merkleRoot != bytes32(0)` and `feeBandCount > 0` which will never be true since `merkleRoot` and `feeBandCount` can never be 0 as can be seen in the `initialize`, `setMerkleRoot` and `setFeeBands` functions.

```
    function canAcceptClaims()
        external
        view
        returns (
            bool ready,
            bool hasRoot,
            bool hasFeeBands,
            bool isWhitelisted,
            bool tokensMatch
        )
    {
        hasRoot = merkleRoot != bytes32(0);
        hasFeeBands = feeBandCount > 0;

        // When directClaimEnabled, spend contract is bypassed for spend
allocations
        // so whitelist check is not required for claims to succeed
        isWhitelisted = directClaimEnabled ||
spend.whitelisted(address(this));

        // Token consistency should always be verified - this is a
configuration check
        tokensMatch =
            (stakingPool.stakingToken() == address(token)) &&
            (spend.token() == address(token));
//....
```

The same could have been done for `tokensMatch` check, however, Spend contract can update the token after deployment via `setToken` function. Since, the token is intended to be the same across all contracts, this `setToken` function is probably not needed and can be removed.

Recommendations

Remove the unnecessary checks.